Compiler Optimizations for Attention and Functional KV Caches

CS6886: Systems Engineering for Deep Learning 1 November 2026

Nishanth Senthil Kumar (EE23B049) IIT Madras

1 Problem, background and approach

Project goal. Reduce unnecessary attention computation and KV-cache movement in IREE using C++ compiler passes. The strongest model result is **26.46% lower latency** for a custom two-candidate SmolLM2 step at context 2048. A separate synthetic forwarding test reduces DRAM traffic by **99.92%**. The two numbers describe different experiments.

What attention and the KV cache do

For each new token, attention forms a *query* Q, compares it with earlier tokens' *keys* K, and uses those scores to mix their *values* V. Softmax turns the scores into weights that sum to one. A KV cache saves earlier K/V vectors so they need not be recomputed for every generated token:

```
# One decoder layer, schematic single-token step:
k_new, v_new = key_projection(token), value_projection(token)
K_next = concat(K_history, k_new)
V_next = concat(V_history, v_new)
out = softmax(Q @ K_next.T / sqrt(head_width)) @ V_next
```

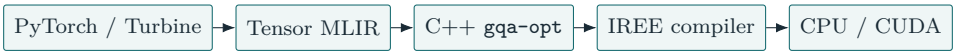
A head is one attention channel. GQA lets several query heads share a smaller set of K/V heads [1, 2]. FP32 and FP64 mean 32-bit and 64-bit floating-point numbers. Cache size grows with the number of stored tokens, layers and KV heads. Avoiding repeated K/V computation and moving fewer history bytes can therefore matter during decoding.

A new tensor value does not always mean a new allocation

The code above describes the values a program must produce. It does not decide how buffers are allocated. A compiler may reuse a buffer, fuse operations or avoid a temporary entirely. Appending one token is not automatically a full-cache copy. Our investigation checks the lowered program before claiming that a copy exists.

Branching creates a useful harder case: two candidate tokens share one history, but each proposes different new rows. Both candidates are evaluated and one is selected. In the model test we take the two most likely next tokens, evaluate each with the shared history, then choose using its likelihood together with that of its best following token (the sum of their log probabilities). The compiler must preserve the shared history while deciding which temporary versions and selections are actually needed. Ordinary greedy decode has no such winner selection.

Where our implementation runs



MLIR is a structured intermediate form of the program; a *pass* rewrites that form. Our C++ patterns inspect operation types, shapes, indices and users, then build equivalent operations. Unsupported patterns are left unchanged. The standalone driver uses MLIR 19; IREE 3.13 consumes its output and produces CPU/GPU executables [12, 17]. This is compiler rewriting, not a hand-written replacement GPU kernel.

What the measurements mean

Claim	Evidence needed
Logical expansion removed	The intermediate program no longer creates the repeated/full temporary tensor.
Allocation removed	Runtime allocation measurements show fewer or smaller buffers.
DRAM traffic reduced	GPU hardware counters show fewer bytes read/written to device memory.
Execution faster	Separate unprofiled timing shows lower latency; compilation is excluded.

Prior work and scope. GQA/MQA establish head sharing [1, 2]; read-after-write identities come from classical array reasoning [3, 4]; PagedAttention studies KV sharing and copy-on-write [5]; DeepSeek-V2 describes latent projection absorption and separated rotary coordinates [7]. Our work implements guarded recognition, tests upstream coverage and evaluates these transformations through IREE. Finding that existing flags solve the original repeat pattern led us to investigate cache versions and projection placement instead.

2 Why the two IREE flags remove the original GQA repeat

Starting graph. GQA shares H_{kv} key/value heads among H_q query heads [1, 2]. For 8 query heads and 2 KV heads, $g = 4$: query heads 0–3 use KV head 0 and heads 4–7 use KV head 1. With batch omitted, $Q : [8, M, D]$ and $K, V : [2, N, D]$:

```
# Natural decomposed attention: expanded tensors have 8 heads.
Kr = repeat_interleave(K, 4, dim=0)   # [8, N, D]
Vr = repeat_interleave(V, 4, dim=0)
P  = softmax((Q @ Kr.transpose(-2, -1)) / sqrt(D), dim=-1)
O  = P @ Vr
```

The saved Linalg IR first repeats K/V across a new sharing axis, giving $[2, 4, N, D]$. It then flattens the two head axes to 8 and calls `linalg.batch_matmul`. A reshape changes how dimensions are viewed; by itself it does not prove that data was copied.

Flag 1: expose matmul as a generic contraction

`--iree-opt-generalize-matmul`

The `GeneralizeLinalgNamedOps` pass replaces the named matrix-multiply operation with `linalg.generic`: an explicit description of its loops and array accesses [13, 14]. The arithmetic is unchanged. A *reduction* adds several products to form one score:

```
# Schematic loop meaning of linalg.batch_matmul(Q, Kr.T):
for h, m, n:                    # parallel output indices
    S[h,m,n] = sum_d(Q[h,m,d] * Kr[h,n,d])
```

Why needed: the tested reshape rules can work through generic operations, but skip named Linalg operations. This flag exposes the loops so later rules can act. It does not remove the repeat by itself.

Flag 2: propagate reshapes through reductions

`--iree-dispatch-creation-enable-aggressive-reshape-movement`

The `BubbleUpExpandShapes` pass moves reshapes through operations. This flag lets it move through supported loops that sum values, rather than only loops whose outputs are independent [13, 14].

Effect here: the flat head number becomes two indices: *KV head* and *query within its sharing group*. Existing fusion then combines the repeated-data producer with the matrix multiply. Its array accesses can read the original K directly:

```
for hk in range(2):
  for j in range(4):
    h = 4*hk + j
    S[h,m,n] = sum_d(Q[h,m,d] * K[hk,n,d])  # no Kr
# The probability-times-V contraction analogously reads V[hk].
```

The result is the proposed rule $h_{kv} = \lfloor h_q/4 \rfloor$. Keeping separate `hk,j` loops may even avoid an integer divide. The full dot product is preserved. These are general loop/reshape/fusion rules, which explains why a search for a GQA-specific optimization did not reveal them.

Evidence and implication for our original pass

Generalize matmul	Aggressive reshape	Broadcast calls	All dispatch calls
Off	Off	2	5
On	Off	2	5
Off	On	2	5
On	On	0	3

O3 CPU compile-to-Flow check: $H_q = 8, H_{kv} = 2$. Pass order: generalization, reshape movement, then fusion. The final dot products read compact $[2, N, D]$ K/V. Saved IR and a broader head-ratio/shape ablation are in the repository.

Why we changed direction. Our original pass builds grouped matrix multiplies directly. Both flags plus fusion already remove the tested repeats, so its 1.86× gain over default CPU lowering is not an extra gain over strong IREE. O3 alone was insufficient. We therefore moved to cache forwarding, commit placement and projection placement. This flag study does not cover every GQA graph: other users of a repeated tensor may still require it.

3 Functional-cache transformations: what changes and why

Examples are pseudocode, not a replacement model implementation. `update(C,p,U)` means "C with rows $p : p + t$ replaced by U. C has P cache rows, H KV heads and D values per head; U has t new rows. The old C must remain valid if the program still uses it.

3.1 Conditional read forwarding: avoid building an unused full version

Before: each candidate forms an updated cache, but downstream sparse attention only gathers a few rows.

```
T = update(C, p, U)             # logical full updated version
R = gather(T, ids)
# After: conditional loads from original history or the patch.
R = stack([U[i-p] if p <= i < p+t else C[i] for i in ids])
```

Example: with 10000 cached rows and a one-row update at 9000, reading `[0,9000,9999]` gives `[C[0],U[0],C[9999]]`. If every user of T is a supported read, T may never need a full buffer. **Use case:** branches read small subsets of a large shared pool. **Limit:** a required full-cache output still has to exist. Ordinary dense attention reads the new row too; forwarding does not remove that attention work. The read-after-write identity is established array algebra [3, 4].

3.2 Masked gather forwarding: a different implementation of the same law

Instead of per-element conditional control flow, gather a compact base and patch and select their elements. The implemented masked variant uses a safe patch index even outside the update:

```
inside = (p <= ids) & (ids < p+t)
safe   = where(inside, ids-p, 0)  # U must have at least 1 row
base   = gather(C, ids)
patch  = gather(U, safe)
R      = where(expand_mask(inside), patch, base)
```

Why the safe index matters: both sides of a selection may be evaluated. Even a value later discarded must be loaded from a valid address. **Tradeoff:** two gathers and a mask add work, so a cost check rejects dense demand. An irregular-shape CUDA case fails to compile in the masked form and uses the conditional fallback. The two forms are alternatives, not gains to add together.

3.3 Winner-cache factoring: select the small update before commit

Two candidates share the same history but propose different new rows. The final choice is a scalar Boolean:

```
# Before: selection consumes two full appended cache values.
Ta, Tb = concat(C, a), concat(C, b)
committed = select(winner_is_a, Ta, Tb)
# After: move the same selection to the small appended rows.
committed = concat(C, select(winner_is_a, a, b))
```

With 4096 old rows, this selects between one-row updates instead of two 4097-row versions. Equal-region overwrite forms are also supported. The pass requires the same history, append axis, compatible shapes/types and one Boolean decision for the entire cache. Different histories or per-element decisions are rejected. **Use case:** committing the winning branch. The model has 60 matches (K and V in 30 layers); ordinary decode has none. Both candidates still run attention, so their cache work is not all eliminated.

3.4 Adjacent-update coalescing: build one small patch

```
# Before: intermediate versions are not observed elsewhere.
C1 = update(C, 100, a)           # a has 1 row
C2 = update(C1, 101, b)          # b has 1 row
# After:
C2 = update(C, 100, concat(a, b))
```

Use case: several adjacent token updates. Combining them can expose a single update that the forwarding pass understands. Updates must touch neighboring, non-overlapping rows, use compatible types and have no other reader of the intermediate version. Large patches are rejected. Coalescing alone was slightly slower in the tested CPU chains; the 2.17–2.34× gains require coalescing *plus forwarding*.

3.5 Gather-index normalization: preserve the right batch coordinates

For source $C : [P, H, D]$ and indices $[F, N, 1]$ (F forks, N reads each), the last1 is an index-component axis, not another batch axis. The required read is $R[f, n, h, d] = C[\text{ids}[f, n, 0], h, d]$.

```
ids2 = reshape(ids, [F, N])      # remove ONLY the unit component
R    = gather(C, ids2)           # preserve f and n coordinates
```

The final size-one axis stores one coordinate, not another batch. Removing only that axis keeps the fork and selected-row coordinates correct. The pass checks this exact depth-one layout and its output shape. It repairs wrong indexing before vectorization; it is not a separate head-sharing speedup.

4 Ownership, projection and numerical transformations

4.1 Snapshot protection and borrowed-buffer copy-on-write

```
# Tensor contract: return the OLD C alongside an updated-cache result.
T = update(C, p, U)
return attention(T), C
# Required ownership if bufferized T would alias borrowed C:
T_buf = clone(C_buf); T_buf[p:p+t] = U
return attention(T_buf), C_buf
```

A *snapshot* is the old cache value that must remain unchanged. Updating its buffer in place would corrupt that result. Before buffer allocation we mark the old observation so it cannot be discarded; at the supported Stream lowering stage we detect borrowed input buffers and copy before writing when needed. If forwarding removes T, that copy may become unnecessary. This is a correctness requirement, not a reason to copy every cache. Incorrect stock results are excluded from speedup baselines. KV sharing/copy-on-write has systems precedents [5, 6].

4.2 Latent projection placement: project the query instead of every key

Let $C : [N, r]$ be an already-compressed cache, $W_K : [r, d]$, and $q : [1, d]$.

```
# Before: project all N cached vectors, then compare with q.
K = C @ Wk                      # [N, d]
scores = q @ K.T
# After: one query projection and a latent-width comparison.
scores = (q @ Wk.T) @ C.T        # [1, r] @ [r, N]
```

For 10000 cached vectors, latent width 64 and key width 512, the first version projects every old vector from 64 to 512. The second projects the current query from 512 to 64 once, then compares in the smaller space. Values similarly use $A(CW_V) = (AC)W_V$: mix compressed vectors first, then project the result. **Use case:** an already-compressed cache, not arbitrary ordinary KV caches. Shape/use and cost checks reject unsupported or expensive cases. Floating-point rounding can change, so reassociation needs a numerical contract. MLA absorption is established prior work [7].

4.3 Partial-RoPE factoring: absorb content while retaining rotation

RoPE encodes position by rotating pairs of coordinates [8]. Moving a projection across that rotation is generally wrong. When the graph explicitly separates content and rotary coordinates, transform the content part and leave the positional part intact [7]:

```
# Before: qC has 512 content coordinates, qR has 32 rotary ones.
s = concat(qC, qR) @ concat(C @ Wk, kR).T
# After: C has 64 latent coordinates; rotate/retain kR as before.
s = (qC @ Wk.T) @ C.T + qR @ kR.T
P = softmax(s / sqrt(512 + 32))  # same scale in both forms
```

A dot product of two concatenated vectors is the sum of their two dot products. The original 544-coordinate score still needs the original scale $\sqrt{512 + 32}$; the smaller internal latent width does not change it. The pass recognizes supported split/concat graphs, not arbitrary position encodings. The reported gains include projection placement; they do not isolate a RoPE-only speedup.

4.4 Native-attention phase guard and decomposed fallback

Online softmax processes tokens in blocks without storing all scores. Each score still needs its full feature sum; partial feature sums are not extra tokens [10, 11]. In the 256-feature example, one 16-wide tile contributes scores $[\log 9, 0]$; the other 15 contribute $[0, 0]$, with values $[1, 0]$:

$$\text{correct} = \frac{9}{9+1} = 0.9, \qquad \text{wrong feature-as-token merge} = \frac{9+15}{9+1+30} = 0.6.$$

The guard prevents the supported CPU schedule from splitting this feature sum incorrectly. The fallback instead uses ordinary matrix multiplies and softmax. An existing upstream early-decomposition option also works on the tested CPU cases. The guard repairs supported schedules; it does not establish a new attention algorithm or a general GPU fix.

4.5 Bounded trig argument reduction: keep large-phase RoPE accurate

```
# Conceptual repaired path, for 32 < abs(x) <= 8192:
xd = f64(x)
r  = xd - floor(xd/(2*pi) + 0.5) * (2*pi)
y  = sin_f32(f32(r))             # analogously for cos
```

Sine/cosine repeat every 2π. First reducing a large angle to a small one in FP64 avoids some FP32 cancellation, then the existing FP32 trig operation is used. The original FP32 phase is retained. This guarded repair supports small static kernels (≤ 4096 elements) and leaves other inputs on the original path; it is not full Payne–Hanek reduction [9]. It raises model correctness from 5/9 to 9/9 natural cases at the same tolerance. Both timed IREE variants use the same repair, so its accuracy benefit is not counted as the cache transformation speedup.

5 Evaluation protocol

Standalone C++ MLIR 19 rewriting precedes IREE 3.13/LLVM 24 compilation (`89ece87b8985`) [12]. Context means the number of cached tokens. GPU: Tesla T4, driver 580.82.07, FP32, TF32 off. SmolLM2-135M-Instruct [15], checkpoint `12fd25f77366`: 30 layers, 9Q/3KV heads, width 64. Both IREE variants use O3, both GQA flags, identical split-argmax settings, trig repair and f64-preservation flags. Timing excludes compilation, prefill and upload: resident outputs, context synchronization, randomized order, 31 samples/repetition; median of nine repetition medians (three prompts, three repetitions). Nsight Compute 2025.3.1 app-range counters [16] use prompt 0, three independent processes and three calls/range, separately from timing. All 112 final counter exports were rechecked.

6 Winner factoring: pretrained two-candidate step

Context	Base ms	Opt. ms	Speedup	Latency lower	Total DRAM change
2048	43.264	31.815	**1.36**×	**26.46%**	−3.91%
3072	63.326	60.386	1.05×	4.64%	+1.24%
4096	82.409	77.858	1.06×	5.52%	−0.27%

This comparison isolates concat winner factoring within the repaired IREE pipeline. All natural and forced-winner cases pass, with exact cache prefixes and unchanged inputs. Worst absolute error: 5.05×10^{-5} (atol= 5×10^{-4}, rtol= 3×10^{-3}). Ordinary decode has no matches. Writes fall but reads rise: full-model DRAM reduction is inconsistent. At context 2048, kernels increase 987→1047/call and logical memcpy remains 188.8 MB/call.

7 Per-transformation operator results

Speedup = baseline time / optimized time: 2× halves latency, while below 1 means slower. CPU rows are separate synthetic studies, not full-model gains. Suite figures use a geometric mean, averaging the ratios on a multiplicative scale. Earlier T4 projection timings include output downloads; final model timings leave outputs resident. Combined gains are not additive. Full per-case times, variant names and source paths: `report/study_results/`.

Transformation	Measured effect	Workload / attribution [17]
Original GQA head reuse	1.86× CPU	Default lowering only; no extra gain established over both upstream flags plus fusion.
Conditional read forwarding	2.04× CPU	Cache A1: 187.10→91.74 μs; same best-correct baseline.
Masked gather forwarding	2.52× CPU	Same A1: 187.10→74.36 μs; alternative to conditional reads.
Winner factoring alone	1.38× CPU	2607.34→1885.37 μs; with masked forwarding: 723.42 μs, **3.60× combined**.
Adjacent-update coalescing	0.94–0.96× alone	4–16-token CPU chains. **2.17–2.34× combined** with masked forwarding; merging enables a later match.
Snapshot protection / COW	2.58× combined	109.01→42.27 μs CPU: protection *plus* masked forwarding vs fastest correct baseline; not protection alone.
Key/value projection placement	3.93× CPU; 1.20× T4	16 cases per platform vs strong IREE before projection absorption. One decode ablation: key-only 1.92×, value-only 1.95×.
Partial RoPE + projection	11.40× CPU; 1.50× T4	Four cases per platform; combined pipeline, no isolated RoPE gain. MLA absorption is prior art.
Correct contraction fallback	10.63× CPU	One wide-head case: 131.11→12.33 μs vs correct guarded fused lowering; not a gain from the guard itself.
Gather normalization / native guard	Correctness enablers	No independent valid speedup established. Wrong stock outputs are excluded from performance baselines.
Stable trig reduction	5/9→9/9 cases pass	Earlier matched-valid-case latency overhead: +0.07% (A), +0.25% (B); not a speed optimization.
Extra aggressive fusion flag	0.97× at 2048	Earlier repaired B run: 32.02→33.16 ms; does not consistently help.

8 Forwarding on T4: mechanism and negative control

Track B fixture	Base ms	Opt. ms	Speedup	DRAM MB: base → opt.
Small sparse forks	0.288	0.246	1.17×	5.831→0.128 (97.80% less)
Large sparse forks	1.777	1.230	1.44×	178.897→0.137 (99.92% less)
Dense no-rewrite control	0.545	1.268	**0.43**×	0.285→0.275 (small / variable)

Conditional forwarding, separate from the model factoring result. Track A large fixture: 1.14×, 178.228→0.132 MB in a second T4 session. Synthetic checks use independent FP64 NumPy references (atol= 5×10^{-5}, rtol= 2×10^{-3}). The unexplained no-rewrite regression and uncontrolled clocks limit causal attribution.

Limitations. One FP32 small model and GPU family; custom one-step branching, not complete serving throughput. Removing a tensor operation, saving a buffer, reducing memory traffic and lowering latency are separate results. Incorrect ownership/numerical cases and irregular masked CUDA compile failures remain recorded. No all-GQA, mixed-precision or global-novelty claim is made.

Prior work. GQA, MLA absorption and decoupled RoPE have identifiable originating papers below. Read-over-write has classical foundations. For selection factoring, adjacent-update merging and this exact compiler repair, no definitive first paper is established here. Cache-sharing papers are related systems precedents, not attribution for our precise MLIR patterns.

References and attribution

[1] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón and S. Sanghai. *GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints*. EMNLP, 2023, pp. 4895–4901. Originating GQA reference. https://aclanthology.org/2023.emnlp-main.298/

[2] N. Shazeer. *Fast Transformer Decoding: One Write-Head is All You Need*. 2019. Introduces multi-query K/V sharing. https://arxiv.org/abs/1911.02150

[3] J. McCarthy. *Towards a Mathematical Science of Computation*. Information Processing'62, 1962, pp. 21–28. Classical functional-array foundation; not a GQA optimization paper. https://www-formal.stanford.edu/jmc/towards/towards.html

[4] C. Tinelli / SMT-LIB. *ArraysEx: Functional Arrays with Extensionality*. Formal select/store read-over-write axioms. https://smt-lib.org/theories-ArraysEx.shtml

[5] W. Kwon et al. *Efficient Memory Management for Large Language Model Serving with PagedAttention*. SOSP, 2023. KV sharing and copy-on-write precedent; not our compiler factoring pass. https://arxiv.org/abs/2309.06180

[6] S. Wang, R. Ren and L. Gui. *ForkKV: Scaling Multi-LoRA Agent Serving via Copy-on-Write Disaggregated KV Cache*. 2026. Related branching-cache system, not the first array-law reference. https://arxiv.org/abs/2604.06370

[7] DeepSeek-AI et al. *DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model*. 2024. Secs.2.1.2–2.1.3 explicitly describe MLA weight absorption and decoupled RoPE. https://arxiv.org/abs/2405.04434

[8] J. Su et al. *RoFormer: Enhanced Transformer with Rotary Position Embedding*. First arXiv version, 2021. Originating RoPE reference. https://arxiv.org/abs/2104.09864

[9] M. H. Payne and R. N. Hanek. *Radian Reduction for Trigonometric Functions*. ACM SIGNUM Newsletter 18(1), 1983, pp. 19–24. Seminal argument-reduction reference; our bounded f64 scheme is not its full algorithm. https://doi.org/10.1145/1057600.1057602

[10] M. Milakov and N. Gimelshein. *Online Normalizer Calculation for Softmax*. 2018. Online normalization background. https://arxiv.org/abs/1805.02867

[11] T. Dao, D. Y. Fu, S. Ermon, A. Rudra and C. Ré. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. NeurIPS, 2022. Fused IO-aware attention background, not implemented anew in this project. https://arxiv.org/abs/2205.14135

[12] C. Lattner et al. *MLIR: A Compiler Infrastructure for the End of Moore's Law*. 2020. https://arxiv.org/abs/2002.11054

[13] IREE contributors. *GeneralizeLinalgNamedOps; BubbleUpExpandShapes; compilation pipelines*. Source pinned to compiler commit `89ece87b89852100c0a4ac3d515e14842d090967`. Implementation evidence for existing transformations, not a GQA-specific paper. Pinned source: GeneralizeLinalgNamedOps.cpp and BubbleUpExpandShapes.cpp.

[14] IREE contributors. *MLIR pass reference: GlobalOptimization and DispatchCreation*. Named-op generalization, reshape propagation through reductions and elementwise fusion. The observed behavior is pinned by the source/IR evidence, since online documentation can change. https://iree.dev/reference/mlir-passes/GlobalOptimization/ https://iree.dev/reference/mlir-passes/DispatchCreation/

[15] Hugging Face. *SmolLM2-135M-Instruct model card and checkpoint*. Pinned revision in the evaluation protocol. https://huggingface.co/HuggingFaceTB/SmolLM2-135M-Instruct

[16] NVIDIA. *Nsight Compute Profiling Guide*. Application-range replay, metric coverage and cache/clock controls. https://docs.nvidia.com/nsight-compute/ProfilingGuide/

[17] CS6886 project. *Attention and KV-cache compiler optimizations: implementation, raw results and reproduction*. Experiment-source snapshot `71cc40ae53c5e60f2a690e67ffbc64e85969e64f`. https://github.com/lazarsalt1441/cs6886_project